ClockScore: Quantifying Deployment-Generalization Gaps in Agent Memory Prediction Tasks

Anonymous
Anonymous Institution
Anonymous, Anonymous

ABSTRACT

Agent memory systems are evaluated using supervised prediction tasks, but the standard methodology of random cross-validation does not reflect deployment conditions where training data precedes test data in time. We define ClockScore, a metric that quantifies the deployment-generalization gap: the mean absolute difference between random-CV performance and sliding-window temporal performance across multiple cut-offs. We evaluate ClockScore on three datasets spanning different task types and memory system architectures. On a production supersession prediction task (2,635 records), ClockScore is 0.103, indicating moderate deployment sensitivity. On a LoCoMo-derived event task (2,541 observations), ClockScore is 0.035, indicating temporal stability. On a controlled synthetic dataset designed for maximal instability (4,000 records), ClockScore is 0.421. The metric correctly orders all three tasks in the expected direction. We further show that different memory system architectures exhibit fundamentally different temporal behavior: a system with supersession chains creates temporally dependent data (nonzero ClockScore), while a system with LLM-based memory extraction converges to a fixed-size memory store, suppressing temporal dependency. ClockScore is general, computable from standard evaluation logs, and available as an open-source Python package.

CCS CONCEPTS

• **Computing methodologies** → **Artificial intelligence**; • **Information systems** → *Data management systems.*

KEYWORDS

agent memory, temporal validation, evaluation methodology, benchmark

1 INTRODUCTION

Agent memory systems are increasingly evaluated using supervised prediction tasks: predicting which memories will become obsolete, which will be retrieved, or which will influence downstream decisions. The standard evaluation methodology uses random cross-validation, which assumes that training and test data are exchangeable. This assumption fails for event-dependent tasks where the outcome depends on future agent behavior: training on data from time period T_1 and testing on data from a later period T_2 produces different results than random splits because the event distribution changes between periods.

Existing evaluation frameworks do not quantify how much a task's performance degrades under temporal splitting. A researcher who reports F1 = 0.7 under random CV has no way to know whether the same model would achieve F1 = 0.7, 0.5, or 0.3 in deployment where training must precede testing. We address this gap by defining ClockScore.

2 DEFINITION

Definition 1 (ClockScore). *Given a dataset $\mathcal{D}$ with n temporally ordered records and a prediction task with metric M (e.g., F1), ClockScore is defined as:*

$$ClockScore(\mathcal{D}) = \frac{1}{W} \sum_{w=1}^{W} |M_{random} - M_{temporal}(w)|$$

where M_{random} is the metric under k-fold random cross-validation, $M_{temporal}(w)$ is the metric when training on $[0, s_w)$ and testing on $[s_w, s_w + \delta)$ (the w-th temporal window), W is the number of valid temporal windows, and s_w and δ are the window start and size parameters.

Property 1 (Bounded). $0 \leq ClockScore \leq 2 \cdot \max(M)$ *for any bounded metric M.*

Property 2 (Monotone in Non-Stationarity). *If the data-generating process is stationary (event distribution does not change over time), ClockScore $\to 0$ as $W \to \infty$.*

Property 3 (Task Discrimination). *ClockScore is higher for event-dependent tasks (where the outcome depends on future agent behavior) than for content-determined tasks (where the outcome depends only on the content itself), because event-dependent tasks exhibit temporal non-stationarity that random CV masks.*

3 IMPLEMENTATION

ClockScore is implemented as a Python package with three components:

3.1 Temporal Window Generator

The window generator creates W temporal splits from a chronologically ordered dataset. Each split trains on $[0, s_w)$ and tests on $[s_w, s_w + \delta)$. We default to $s_{\min} = 0.3$, $s_{\max} = 0.8$, $W = 8$, and $\delta = 0.1n$. Windows with fewer than 10 test positives are skipped.

3.2 Metric Computation

The metric M defaults to macro-averaged F1. For each window, logistic regression with fold-level standardization is trained on the training partition and evaluated on the test partition. Random-CV baseline uses stratified 5-fold with group-aware splitting.

3.3 Uncertainty Quantification

ClockScore reports block-bootstrap confidence intervals using 2,000 resamples with 95% percentile CIs. Blocks are automatically detected from metadata.

The package accepts JSONL and CSV formats and requires no modifications to the prediction model.

4 EVALUATION

4.1 Datasets

We evaluate ClockScore on three datasets:

Table 1: Dataset characteristics.

Dataset	Records	Positive	Task Type
MemTether	2,635	386	Event-dependent (supersession)
LoCoMo Event	2,541	832	Content-determined (future event)
Controlled Synth.	4,000	1,800	Designed unstable

4.2 Results

Table 2: ClockScore results across three datasets.

Dataset	ClockScore	M_{random}	Ordering
Controlled Synth.	**0.421**	0.469	1 (most unstable)
MemTether	**0.103**	0.150	2 (moderate)
LoCoMo Event	**0.035**	0.347	3 (most stable)

ClockScore correctly orders all three tasks in the expected direction.

Controlled synthetic (0.421). This dataset contains 2,000 entity-keyed records (200 entities × 10 writes) and 2,000 entity-free records (200 topics × 10 writes), with identical source, type, confidence, and time distributions. Under an entity-keyed deterministic rule, 90% of entity-anchored records are superseded. Under an exact-duplicate-only rule for entity-free records, the rate is 0%. The temporal windows capture a sharp transition: early windows (before the last writes in each group) show high F1 because most positives are present; late windows show near-zero F1 because the remaining records are entity-free (no supersession events). This creates maximal ClockScore by construction.

MemTether supersession (0.103). The production dataset exhibits a burst pattern: 86.8% of supersessions on September 25, 0% after September 28. Temporal windows that include the burst show high F1; windows after the burst show near-zero F1. The moderate ClockScore reflects this non-uniform event distribution rather than a systematic degradation.

LoCoMo event (0.035). The LoCoMo-derived event task asks whether an observation becomes a future event in a later session. The 832 positives are distributed across 10 conversations and 272 sessions without a dominant burst. Temporal windows produce consistent F1 values (0.277–0.370), yielding a low ClockScore. This confirms that content-determined tasks are temporally stable.

4.3 Cross-System Behavioral Differences

4.4 Cross-System Behavioral Differences

We additionally compare the temporal behavior of two memory system architectures by feeding the same data into each:

Table 3: Memory system behavioral differences (mem0 vs. supersession-based).

Behavior	Supersession	mem0
New memories created	Every write	LLM decides
Old memories	Superseded (kept)	Overwritten/merged
Memory growth	Unbounded	Converges to ceiling
Temporal dependency	High	Suppressed
ClockScore implication	Nonzero	Near zero

We deployed mem0 (v2.0.20, OpenAI-compatible LLM + local sentence-transformer embeddings) with a production memory subset and conducted a temporal shadow test.

Setup. We ingested 30 chronologically early records from a 30-day production deployment (15 AI agents) into mem0. After confirming 20 memories were created, we added 30 additional records from a later time period.

Result. All 20 original memories survived. Zero were overwritten. Zero new memories were created. mem0's LLM-based extraction classified all new content as semantically equivalent to existing memories, capping the store at 20 entries.

Comparison. A supersession-based system (MemTether) creates a new record per write with an explicit supersession chain. After 30 days, the store grew to 2,635 records with 347 chains. Every write is preserved with temporal ordering, creating strong temporal dependency.

Implication. Supersession-based systems produce data with high temporal dependency (higher ClockScore). LLM-extraction systems converge to a fixed-size store (lower ClockScore). Neither is superior, but they require different evaluation approaches. ClockScore makes this visible.

Table 4: Temporal shadow test on mem0.

Metric	Value
Original memories	20
Records added	30
Originals overwritten	0
New memories created	0
Store growth	0%

4.5 Statistical Validation

We use block bootstrap (2,000 resamples) to quantify uncertainty. Blocks are defined by source agent (MemTether) or conversation (LoCoMo), respecting the data's temporal grouping.

5 DISCUSSION

ClockScore addresses a practical problem: a researcher reporting random-CV results for an event-dependent task currently has no way to communicate how much those results would change in deployment. ClockScore provides this information as a single number: ClockScore = 0 means random CV is a valid estimate; ClockScore > 0 means the task has temporal dependency and the random-CV estimate is optimistic.

5.1 Implications for Benchmark Design

Benchmarks that evaluate agent memory systems using prediction tasks should report ClockScore alongside standard metrics. A benchmark with high ClockScore (e.g., > 0.1) is deployment-sensitive: random-CV rankings may not reflect real-world performance. A benchmark with low ClockScore (< 0.05) is temporally stable: random CV is a valid estimate.

5.2 Implications for System Design

The cross-system comparison shows that memory system architecture directly affects temporal dependency. Systems with explicit supersession chains create temporally dependent data (all writes are preserved and ordered). Systems with LLM-based memory extraction converge to a fixed-size store (new content is absorbed into existing memories). Neither design is inherently superior, but they produce different evaluation characteristics that ClockScore makes visible.

6 PRACTICAL IMPLICATIONS

6.1 For Benchmark Designers

Benchmarks that evaluate agent memory systems using prediction tasks should report ClockScore alongside standard metrics. A benchmark with ClockScore > 0.1 is deployment-sensitive: rankings based on random CV may not reflect real-world performance. We recommend that benchmark papers include ClockScore in their evaluation tables, similar to how they currently report confidence intervals.

6.2 For System Developers

If your system produces data with ClockScore > 0.1 on a downstream prediction task, your random-CV evaluation may be overly optimistic. Consider: (1) reporting temporal split results alongside random CV; (2) using the temporal estimate as the primary deployment-aligned score; (3) investigating which features contribute to temporal dependency.

6.3 For Reviewers

When reviewing papers that use prediction tasks to evaluate agent memory systems, ask for ClockScore. If the authors report only random-CV results on an event-dependent task, the reported performance may be inflated relative to deployment conditions. ClockScore provides a single number that quantifies this risk.

7 CLOCKSCORE ACROSS TASK TYPES

ClockScore differentiates three task categories:

Table 5: Interpretation guide.

Range	Interpretation	Example
< 0.05	Stable	Content-based QA
0.05–0.15	Moderate	Event prediction
> 0.15	Unstable	Designed-unstable

Content-determined tasks: outcome depends on content only. ClockScore expected low. Our LoCoMo result: 0.035.

Agent-behavior-determined tasks: outcome depends on future agent writes. ClockScore expected moderate. Our MemTether result: 0.103.

Environment-determined tasks: outcome depends on external state. ClockScore expected high. Our controlled synthetic: 0.421.

7.1 Relationship to Existing Metrics

ClockScore complements standard metrics. F1 measures model quality; ClockScore measures the *reliability* of that quality estimate under deployment conditions. A model can have high F1 and high ClockScore (good but unreliable) or low F1 and low ClockScore (poor but consistent). Researchers should report both.

ClockScore is distinct from concept drift detection (which identifies when distributions change), leakage detection (which identifies features encoding future information), and robustness metrics (which measure input perturbation sensitivity). ClockScore measures temporal reordering sensitivity.

8 LIMITATIONS

- ClockScore depends on the choice of temporal window parameters (W, window size). We use $W = 8$ windows of 10% each, but other parameterizations may yield different values.
- The current evaluation uses three datasets from a single research group. Cross-community validation on external benchmarks is needed.
- The mem0 comparison tests one system configuration; other configurations (different LLM, different extraction prompts) may produce different behavior.
- ClockScore does not distinguish between temporal non-stationarity and data leakage; both inflate the score.

9 CONCLUSION

We define ClockScore, a metric that quantifies deployment-generalization gaps in agent memory prediction tasks. ClockScore correctly orders three datasets of different task types and reveals that memory system architecture directly affects temporal dependency. We release ClockScore as an open-source Python package to enable standardized temporal evaluation across the agent memory community.

REFERENCES

[1] Wu, C. et al. LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory. arXiv:2410.10813, 2024.
[2] mem0ai. mem0: The Memory Layer for AI Agents. GitHub, 2026.
[3] Vectorize.io. Hindsight: Agent Memory That Learns. GitHub, 2026.
[4] Yadav, N. MemStrata: Deterministic Supersession. arXiv:2606.26511, 2026.

[5] Tashman, L.J. Out-of-sample tests of forecasting accuracy. IJF 16(4), 2000.
[6] Bergmeir, C. and Benítez, J.M. On the use of cross-validation for time series predictor evaluation. Info. Sciences 191, 2012.
[7] Kaufman, S. et al. Leakage in data mining. ACM TKDD 6(4), 2012.
[8] Kapoor, S. and Narayanan, A. Leakage and the reproducibility crisis. Patterns 4(12), 2023.
[9] LoCoMo: Long Conversations for Multi-hop reasoning. 2024.
[10] Gama, J. et al. A survey on concept drift adaptation. ACM Comp. Surveys 46(4), 2014.